

Competitive Programming

Section 5: Sorting as a tool

Competitive Programming Course

2026-07-13

Table of contents

1	Why sorting matters	3
2	What sorting changes	4
3	Adjacent after sorting	5
4	Sorting keys	6
5	Tie-breaking	7
6	Comparator discipline in C++	8
7	Interval merging	9
8	The merge invariant	10
9	Sorting and stability	11
10	Integer sizes	12
11	Proof habits	13
12	Common mistakes	14
13	Before you submit	15
14	Exercises	16

1 Why sorting matters

Sorting is not just a way to print things in order. In competitive programming, sorting is a transformation that makes hidden structure visible.

After sorting:

- close values become adjacent;
- equal values form contiguous blocks;
- intervals can be processed from left to right;
- tie-breaking rules become explicit keys;
- greedy choices often become local.

The cost is usually:

$$O(n \log n).$$

For $n \leq 200000$, sorting is usually acceptable. That makes it one of the most common first tools to try after direct linear scans.

2 What sorting changes

Consider the unsorted values:

```
8 1 4 10 6
```

It is hard to see which two values are closest. After sorting:

```
1 4 6 8 10
```

the closest pair must be adjacent. We only need to check:

```
4-1, 6-4, 8-6, 10-8
```

This is the central sorting pattern:

```
global relationship before sorting
  |
  v
local relationship after sorting
```

3 Adjacent after sorting

Why is the minimum absolute difference found among adjacent sorted values?

Suppose the sorted array is:

```
b1 <= b2 <= ... <= bn
```

Take any non-adjacent pair b_i and b_j with $i + 1 < j$. There is at least one value between them, such as b_{i+1} . Then:

$$b_j - b_i = (b_{i+1} - b_i) + \dots + (b_j - b_{j-1})$$

At least one adjacent gap inside that interval is no larger than the whole gap. Therefore a non-adjacent pair cannot be the unique place where the minimum is hidden. Checking all adjacent gaps is enough.

This proof style appears often: sorting lets us argue that any skipped object has a local witness that is at least as good.

4 Sorting keys

Many problems ask for an order that is not plain ascending.

Example:

Sort contestants by higher score first, then lower penalty, then lexicographically smaller name.

This is a lexicographic key:

```
(-score, penalty, name)
```

In Python:

```
rows.sort(key=lambda row: (-row.score, row.penalty, row.name))
```

In C++, write a comparator:

```
sort(rows.begin(), rows.end(), [](const Row &a, const Row &b) {  
    if (a.score != b.score) return a.score > b.score;  
    if (a.penalty != b.penalty) return a.penalty < b.penalty;  
    return a.name < b.name;  
});
```

The comparator is not decoration. It is the formal statement of the ordering.

5 Tie-breaking

Tie-breaking must be deterministic. If the statement says:

1. score descending;
2. penalty ascending;
3. name ascending;

then all three levels matter.

For:

```
alice 300 50
bob   400 90
carol 400 70
dave  300 40
```

the sorted order is:

```
carol bob dave alice
```

carol beats bob because both have score 400, but $70 < 90$. dave beats alice because both have score 300, but $40 < 50$.

6 Comparator discipline in C++

A C++ comparator must define a strict weak ordering. At this level, the practical rule is:

- return **true** only when **a** should come before **b**;
- if all fields are equal, return **false**.

Do not write:

```
return a.score >= b.score;
```

This says that equal elements should come before each other, which breaks the sort contract.

7 Interval merging

Sorting intervals by start point turns a global union problem into a single left-to-right scan.

Intervals:

```
[5,7], [1,3], [3,4], [10,12], [11,20]
```

Sort by start:

```
[1,3], [3,4], [5,7], [10,12], [11,20]
```

For ordinary closed intervals, $[1,3]$ and $[3,4]$ touch at endpoint 3, so they merge into $[1,4]$. Intervals $[5,7]$ and $[10,12]$ do not touch that merged interval. Also $[10,12]$ and $[11,20]$ overlap, so they merge into $[10,20]$.

The result is:

```
[1,4], [5,7], [10,20]
```

8 The merge invariant

During a sorted scan, keep a list of merged intervals. The last interval in that list is the only one that can overlap or touch the next interval.

Why? Because all later intervals start at least as far right as the current one. Earlier merged intervals end before the last merged interval starts, so a new interval cannot jump back and overlap them.

The decision is local:

```
if next_left <= current_right:
    merge
else:
    start a new merged interval
```

This is the ordinary rule for closed intervals: equality means that the two intervals share an endpoint. Some statements instead explicitly say that consecutive integer positions should also be joined. Only in that different convention should you use:

```
next_left <= current_right + 1
```

Always read the statement's interval convention.

9 Sorting and stability

A stable sort preserves the input order of equal keys. Python's `sort` is stable; C++ `sort` is not guaranteed stable.

However, in contest problems you usually should not rely on stability unless the problem explicitly asks for original order among ties. It is clearer to put every tie-breaking field into the key.

If the desired final tie is original index, store the index and sort by it:

```
(-score, penalty, original_index)
```

10 Integer sizes

Sorting itself does not change integer size, but comparisons and derived quantities can overflow.

For minimum gap:

```
1000000000 - (-1000000000) = 2000000000
```

This still fits in signed 32-bit, but many similar tasks use 10^{18} , where `long long` is required. When in doubt, use `long long` for numeric keys and differences.

11 Proof habits

Sorting proofs usually have one of these shapes:

1. **Adjacency proof:** after sorting, any non-adjacent candidate contains an adjacent candidate at least as good.
2. **Order invariant:** after processing sorted items up to position i , the maintained structure correctly summarizes that prefix.
3. **Tie-key proof:** the comparator exactly matches the statement's priority list.

For interval merging, the invariant is:

after processing the first i sorted intervals, the **merged** list is the correct merged union of those intervals.

12 Common mistakes

- Sorting strings lexicographically when the real primary key is numeric.
- Forgetting to reverse only one key, such as score descending but penalty ascending.
- Writing an invalid C++ comparator with `<=` or `>=`.
- Assuming `unordered_map` or unsorted data has meaningful order.
- Checking all pairs after sorting when adjacent checks are enough.
- Forgetting that touching intervals may or may not merge depending on the statement.
- Relying on stable sort instead of writing the tie rule explicitly.

13 Before you submit

For sorting problems, ask:

1. What exactly is the sort key?
2. Which fields are ascending and which are descending?
3. What happens when all explicit keys tie?
4. Does the proof rely on adjacency after sorting?
5. Are intervals closed, half-open, overlapping, or touching?
6. Is the output expected in sorted order?

14 Exercises

The local exercises cover three common sorting uses:

1. `a_min_adjacent_gap`: sort to make closest values adjacent.
2. `b_rank_table`: sort by a multi-level custom key.
3. `c_merge_intervals`: sort intervals and merge in one scan.

Run:

```
python3 sections/05_sorting_as_tool/check.py
```